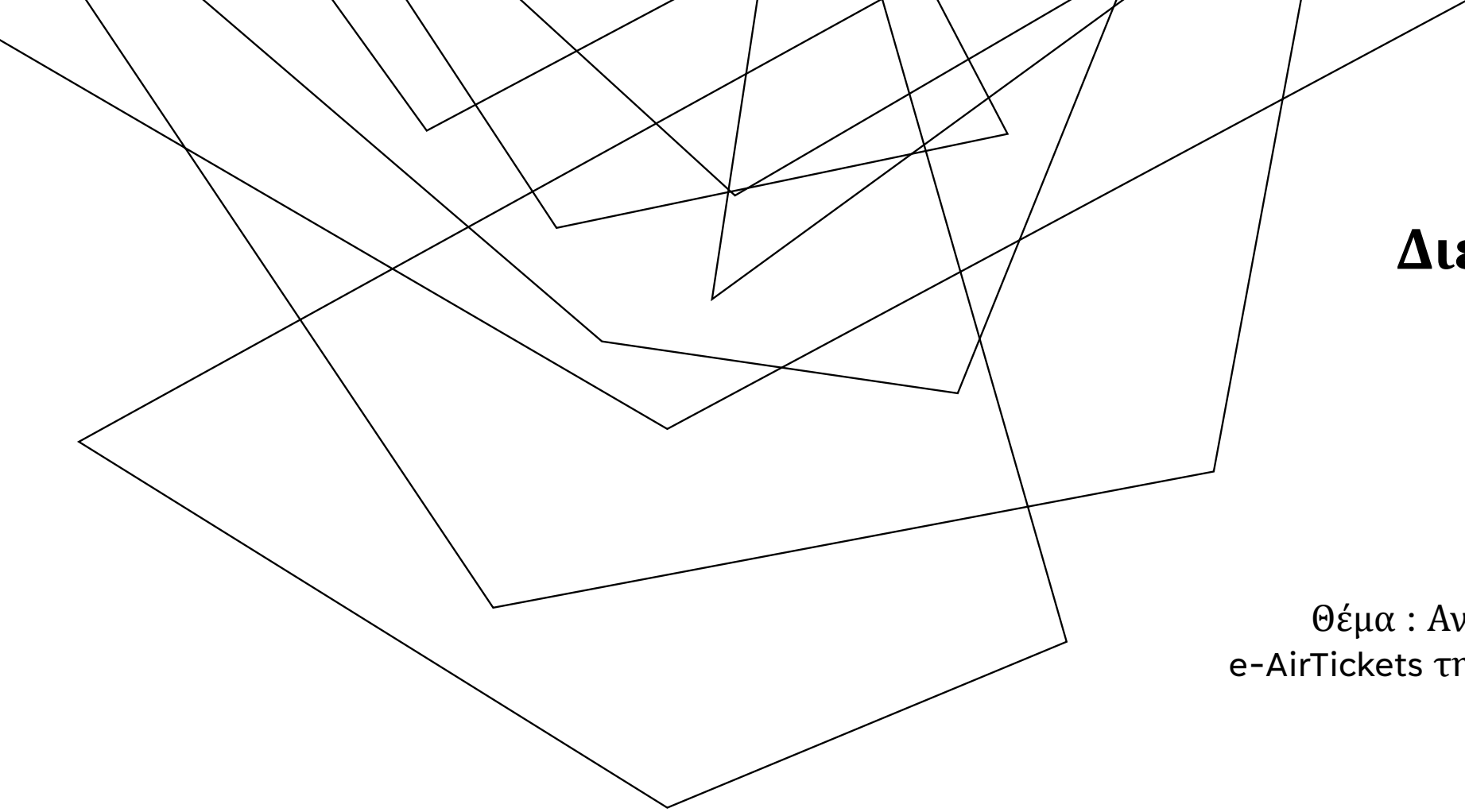




PROJECT **Διεπαφή Χρήστη** **Υπολογιστή**

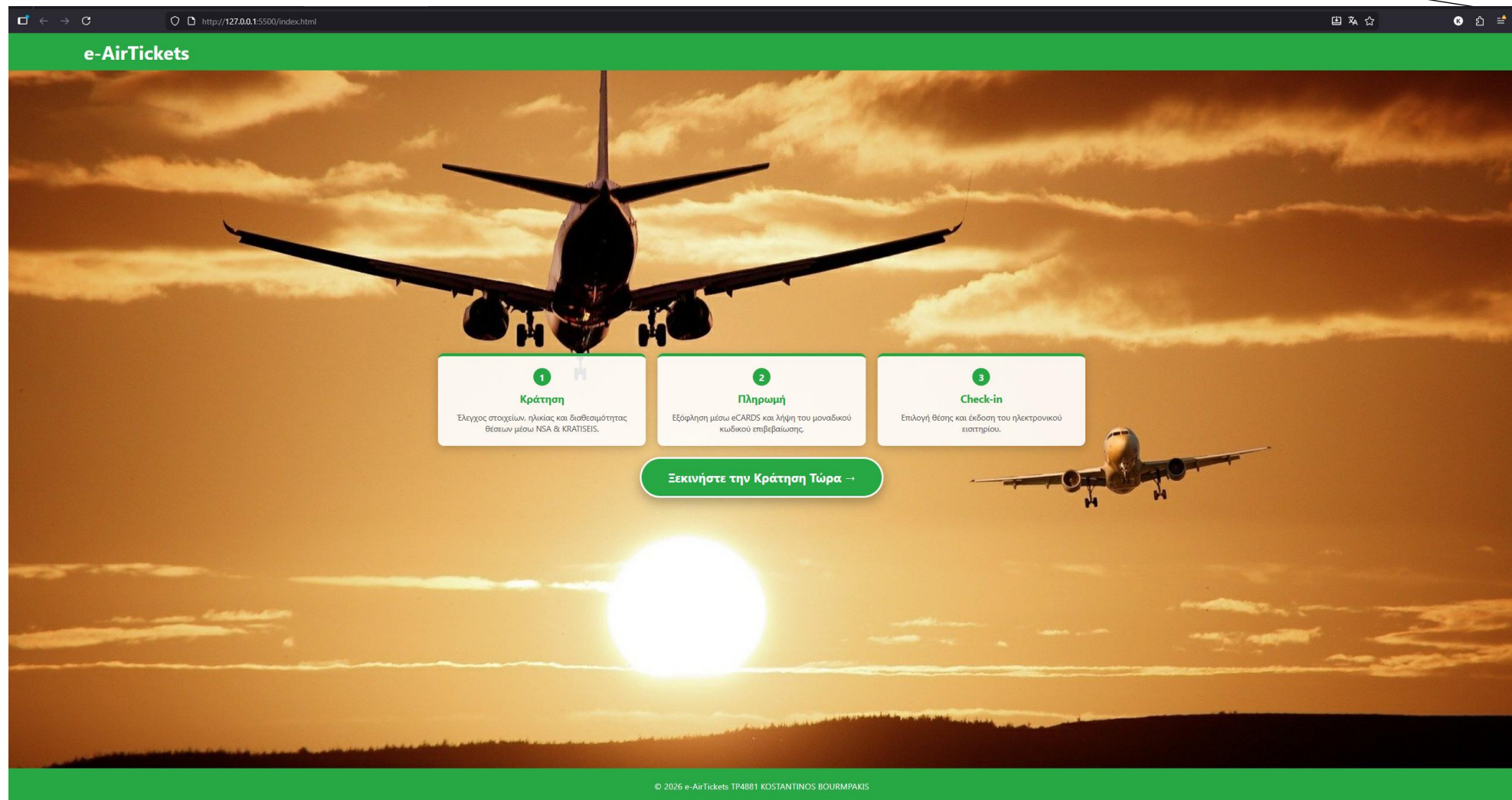
Θέμα : Ανάπτυξη Συστήματος
e-AirTickets της εταιρίας AirContinental

ΚΩΝΣΤΑΝΤΙΝΟΣ ΜΠΟΥΡΜΠΑΚΗΣ ΤΠ4881

- **Στόχος :** Ψηφιοποίηση της διαδικασίας Κράτησης , Πληρωμής και Check-in αεροπορικών εισητηρίων προς μια (συγκεκριμένη) χώρα.
- Ο έλεγχος για τα τρία αυτά συστήματα γίνεται μέσω εξωτερικών Συστημάτων για ευκολία εφόσον εμείς εστιάζουμε στην διεπαφή
Διαχείριση Ταυτοποίησης : Σύστημα NSA
Διαχείριση Δρομολογίων , Κρατήσεων : Σύστημα KRATISEIS
Διαχείριση Πληροφοριών Καρτών : Σύστημα eCARDS
- **Απαιτήσεις :** Έλεγχος ενηλικότητας, εγκυρότητα διαβατηρίου, ακρίβεια 16-ψήφιας κάρτας και δυναμικός υπολογισμός κόστους.

Σχεδίαση Συστήματος

- **Αρχιτεκτονική** : Modular CSS για ταχύτητα φόρτωσης και JavaScript για τη λογική των ελέγχων.
(Βλέπε αρχεία Airtickets/CSS και το αρχείο JS/script.js)
- **Διεπαφή Χρήστη (UI)** : Χρήση Scaled Layout (Flexbox) για προσαρμογή σε κάθε μέγεθος οθόνης ([Εικόνα 1](#))
- **Ροή Δεδομένων**: Χρήση του localStorage για τη μεταφορά τυχαίων κωδικών και κόστους μεταξύ των σελίδων.



Εικόνα 1 : Home page με hint λειτουργίας και το κουμπί έναρξης κράτησης

1. Υποβολή Αίτησης

Όνομα :
KOSTANTINOS

Επώνυμο :
BOURMPAKIS

Ημερομηνία Γέννησης :
09 / 03 / 2000

Αριθμός Πληθυσμίου :
AA123

Διαβατήριό :
AB1234567

Πλήθος Θέσεων (minimum = 1) :
5

Έλεγχος Στοιχείων

127.0.0.1:5500

Θετικό Μήνυμα Ελέγχου Θέσεων! Παρακαλώ εισάγετε τον Ειδικό Κωδικό visa.

OK

ΚΩΝΣΤΑΝΤΙΝΟΣ ΜΠΟΥΡΜΠΑΚΗΣ ΑΜ : ΤΠ4881

1. Υποβολή Αίτησης

Όνομα :
KOSTANTINOS

Επώνυμο :
BOURMPAKIS

Ημερομηνία Γέννησης :
09 / 03 / 2000

Αριθμός Πληθυσμίου :
AA123

Διαβατήριό :
AB123

Πλήθος Θέσεων (minimum = 1) :
5

Έλεγχος Στοιχείων

Έλεγχος Visa (NSA)

123456789

Επιβεβαίωση & Πληρωμή

127.0.0.1:5500

Θετικό Μήνυμα Ελέγχου Ασφαλείας από NSA! Κωδικός Αίτησης: 162806

OK

Στάδια Υλοποίησης

- **Βήμα 1 (Booking)** : Υλοποίηση Regular Expressions και έλεγχος για όλα τα πεδία της φόρμας το οποίο γίνεται με την συνάρτηση [submitBooking\(\)](#)

Με της εντολές `const nameRegex = /^[A-Z]{1,25}$/;`
`const passportRegex = /^[A-Z]{1,2}[0-9]{7,8}$/;`
`const flightNumberRegex = /^[A-Z]{2}[0-9]{6,7}$/;`

Θέτουμε το format που πρέπει να έχουν τα πεδία για να γίνει έπιτα που πάρουμε τα δεδομένα με την `document.getElementById('id της μεταβλητής που θελουμε').value.trim();` (`value.trim()` βγαζουμε τα white spaces)

Και αναλόγως αν είναι σωστά ή λάθος δίνουμε μήνυμα στην οθόνη με την `alert();`

Για τον έλεγχο των θέσεων δημιουργούμε τις θέσεις με την [createAirPlaneMap\(\);](#)

Όπου καλώντας την [toggleSeat\(\);](#) κανουμε reserve random θέσεις για να γίνει ο έλεγχος υπάρχων θέσεων που ζητήθηκαν στην [submitBooking\(\)](#) και να μην μπορεί αργότερα ο χρήστης να τις επιλέξει.

Και αν οι έλεγχοι έχουν περάσει περνάμε στην είσοδο και έλεγχο της Visa με την [finalVisaCheck\(\);](#) που αν όλα είναι σωστά μεταφερόμαστε στην σελίδα της πληρωμής , αλλιώς τερματίζεται η διαδικασία και επιστρέφουμε στην αρχική σελίδα

FUNCTION submitBooking()

```
29
30 function submitBooking() {
31     const nameRegex = /^[A-Z]{1,25}$/;
32     const passportRegex = /^[A-Z]{1,2}[0-9]{7,8}$/;
33     const flightNumberRegex = /^[A-Z]{2}[0-9]{6,7}$/;
34
35
36     const today = new Date();
37
38     const firstName = document.getElementById('firstName').value.trim();
39     const lastName = document.getElementById('lastName').value.trim();
40     const passport = document.getElementById('passport').value.trim();
41     const flightNumber = document.getElementById('flightNumber').value.trim();
42
43     const birthDateValue = document.getElementById('birthDate').value;
44     const birthDate = new Date(birthDateValue);
45     let age = today.getFullYear() - birthDate.getFullYear();
46
47     const requestedSeats = parseInt(document.getElementById('seatsCount').value);
48
49     if (!birthDateValue || isNaN(requestedSeats)) {
50         alert("ΣΥΜΠΛΗΡΩΣΤΕ ΟΛΑ ΤΑ ΠΑΙΔΙΑ.");
51         return;
52     }
53
54     if (!nameRegex.test(lastName) || !nameRegex.test(firstName)) {
55         alert("ERROR: ΕΙΣΑΓΕΤΑΙ ΜΟΝΟ ΚΕΦΑΛΕΟΥΣ ΛΑΤΙΝΙΚΟΥΣ ΧΑΡΑΚΤΗΡΕΣ (έως 25 χαρακτήρες)");
56         return;
57     }
58
59     if (!passportRegex.test(passport)) {
60         alert("ERROR: ΜΗ ΕΓΚΥΡΟΣ ΑΡΙΘΜΟΣ ΔΙΑΒΑΤΗΡΙΟΥ (π.χ. C12345678 ή K1234567).");
61         return;
62     }
63
64     if (!flightNumberRegex.test(flightNumber)) {
65         alert("ΜΗ ΕΓΚΥΡΟΣ ΑΡΙΘΜΟΣ ΠΤΗΣΗΣ (π.χ. AA1234567).");
66         return;
67     }
68
69     if (age < 18) {
70         alert("ΑΠΟΤΥΧΙΑ ΚΡΑΤΗΣΗΣ: Ο ΥΠΟΨΗΦΙΟΣ ΠΕΛΑΤΗΣ ΕΙΝΑΙ ΚΑΤΩ ΤΩΝ 18.");
71         return;
72     }
73
74     if (requestedSeats > availableSeats) {
75         alert("Μη διαθέσιμος αριθμός θέσεων από KRATISEIS.");
76         return;
77     }
78
79     const pricePerSeat = Math.floor(Math.random() * (400 - 70 + 1)) + 70;
80     localStorage.setItem('totalRequestedSeats', requestedSeats);
81     localStorage.setItem('calculatedTotalCost', pricePerSeat * requestedSeats);
82
83
84     alert("Θετικό μήνυμα Ελέγχου θέσεων! Παρακαλώ εισάγετε τον Ειδικό Κωδικό visa.");
85     document.getElementById('visaSection').style.display = 'block';
86 }
```

FUNCTION createAirplaneMap()

```
151 function createAirplaneMap() {
152     const map = document.getElementById('airplaneMap');
153     const required = parseInt(localStorage.getItem('totalRequestedSeats')) || 1;
154     document.getElementById('seatInfo').innerText = "Επιλέξτε ακριβώς " + required + " θέσεις.";
155     map.innerHTML = "";
156     selectedSeatsArray = [];
157
158     for (let i = 1; i <= 10; i++) {
159         ['A', 'B', 'C', 'D'].forEach(l => {
160             const btn = document.createElement('button');
161             btn.className = "seat";
162             btn.innerText = i + l;
163             if (Math.random() < 0.2) { btn.disabled = true; btn.innerText = "X"; }
164             else { btn.onclick = () => toggleSeat(btn, i + l); }
165             map.appendChild(btn);
166         });
167     }
168 }
```


FUNCTION toggleSeat()

```
170 function toggleSeat(btn, seatId) {  
171     const required = parseInt(localStorage.getItem('totalRequestedSeats')) || 1;  
172     if (btn.classList.contains('selected')) {  
173         btn.classList.remove('selected');  
174         selectedSeatsArray = selectedSeatsArray.filter(s => s !== seatId);  
175     } else if (selectedSeatsArray.length < required) {  
176         btn.classList.add('selected');  
177         selectedSeatsArray.push(seatId);  
178     }  
179 }
```

FUNCTION finalVisaCheck()

```
88  function finalVisaCheck() {
89      const visa = document.getElementById('visaCode').value;
90      if (visa.length === 9 && !isNaN(visa)) {
91          alert("Θετικό Μήνυμα Ελέγχου Ασφαλείας από NSA! Κωδικός Αίτησης: " + validAppCode);
92          window.location.href = "payment.html";
93
94          return;
95      } else {
96          alert("Αρνητικό Μήνυμα Ελέγχου Ασφαλείας από NSA. Τερματισμός.");
97          window.location.replace("../index.html");
98
99          return;
100     }
101 }
```

- **Βήμα 2 (Payment)** : Από την αρχή του κώδικα δημιουργούμε με τον παρακάτω κώδικα τους κωδικούς αίτησης κράτησης και check-in ώστε να μπορούμε να κάνουμε τους ελέγχους για το αν θα συνεχίσουμε ή θα τερματίσουμε την διαδικασία χρησιμοποιώντας το localStorage εφόσον δεν έχουμε Βάση δεδομένων

```
1  if (!localStorage.getItem('validAppCode')) {
2    localStorage.setItem('validAppCode', generateAppCode());
3  }
4  if (!localStorage.getItem('validCheckinCode')) {
5    localStorage.setItem('validCheckinCode', generateCheckinCode());
6  }
7
8  const availableSeats = generateAvailableSeats();
9  const validAppCode = localStorage.getItem('validAppCode');
10 const validCheckinCode = localStorage.getItem('validCheckinCode');
11 let selectedSeatsArray = [];
12
13 function generateAvailableSeats() {
14   return Math.floor(Math.random() * (50 - 1 + 1)) + 1;
15 }
16
17 function generateAppCode() {
18   return Math.floor(100000 + Math.random() * 900000).toString();
19 }
20
21 function generateCheckinCode() {
22   const letters = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
23   let randomLetters = letters.charAt(Math.floor(Math.random() * letters.length)) +
24     letters.charAt(Math.floor(Math.random() * letters.length));
25   let randomNumbers = Math.floor(1000 + Math.random() * 9000).toString();
26   return randomLetters + randomNumbers;
27 }
```

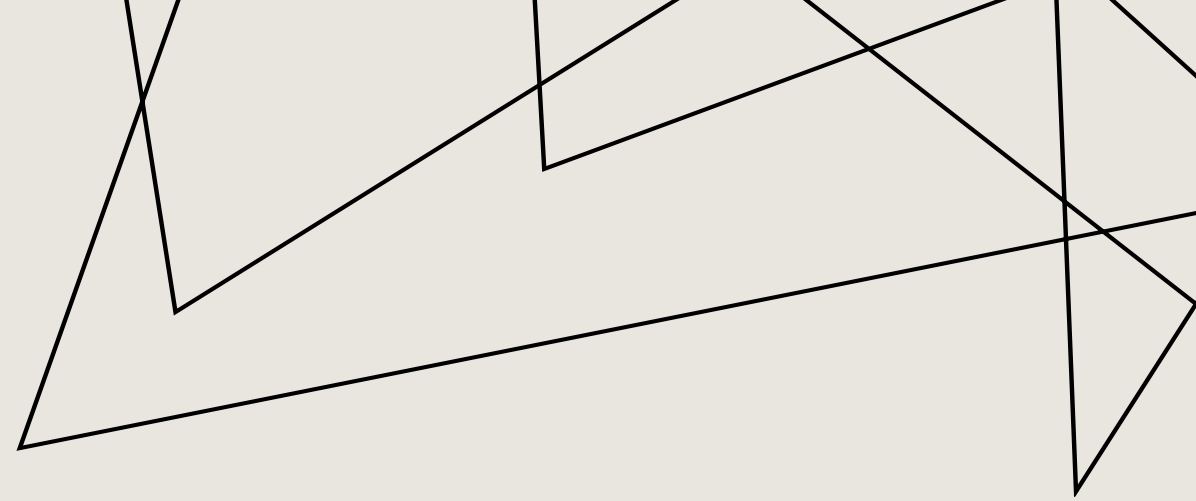
- Αφού γίνει η επιβεβαίωση με την χρήση της [checkApplicationCode\(\)](#);

Προσθέτουμε τα στοιχεία της κάρτας ο χρήστης και το τηλέφωνο για την επιβεβαίωση για να πληρωθεί το συνολικό ποσό που έρχεται από την [submitBooking\(\)](#);

```
79 | const pricePerSeat = Math.floor(Math.random() * (400 - 70 + 1)) + 70;  
80 | localStorage.setItem('totalRequestedSeats', requestedSeats);  
81 | | localStorage.setItem('calculatedTotalCost', pricePerSeat * requestedSeats);  
82 |
```

- Κατά την ολοκλήρωση της διαδικασίας όταν πατηθεί η πληρωμή και μετάβαση στο checkin

τρέχει η [executePayment\(\)](#); για τον έλεγχο των στοιχείων και έπειτα την μετάβαση στο checkin

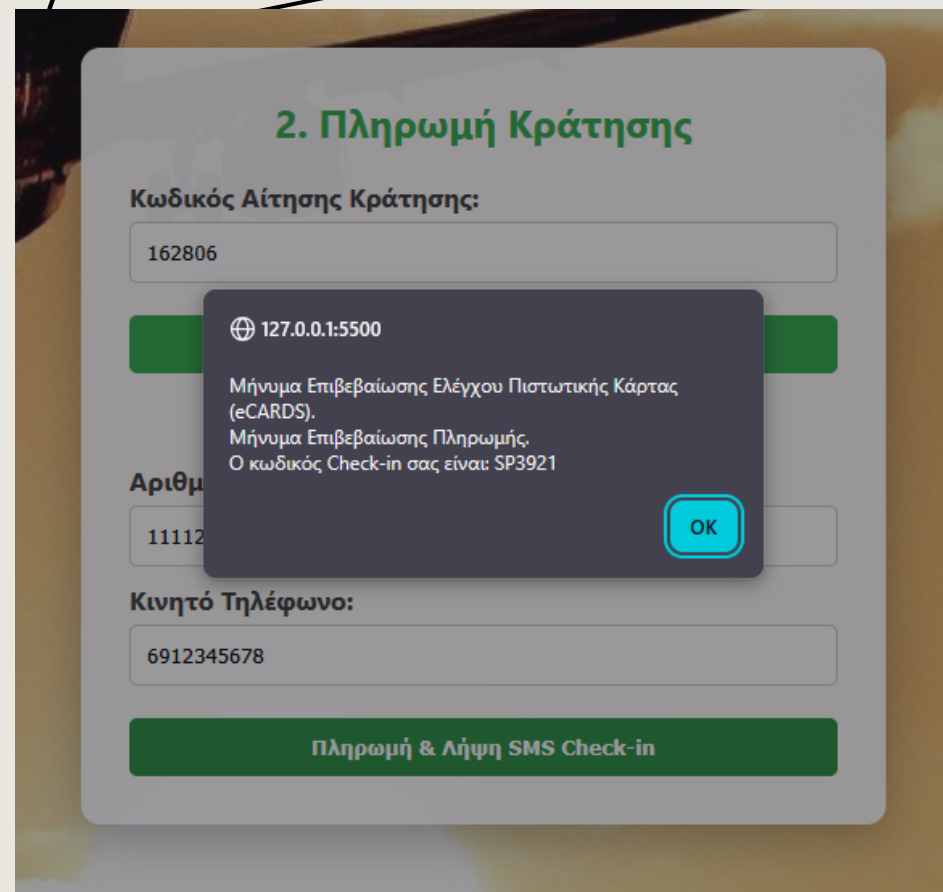
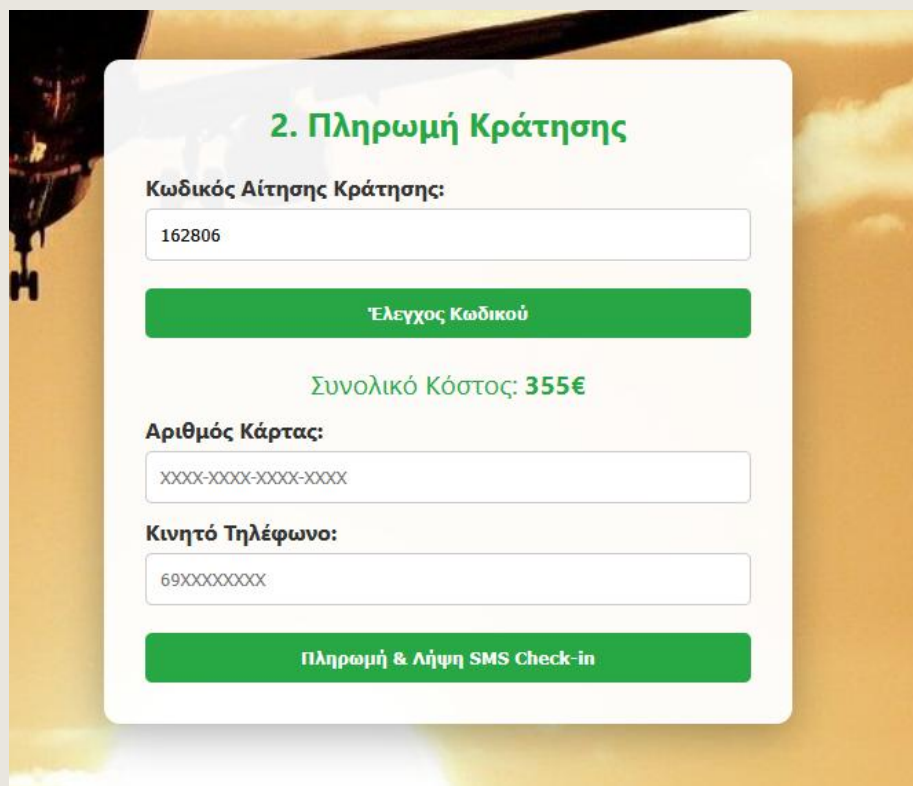
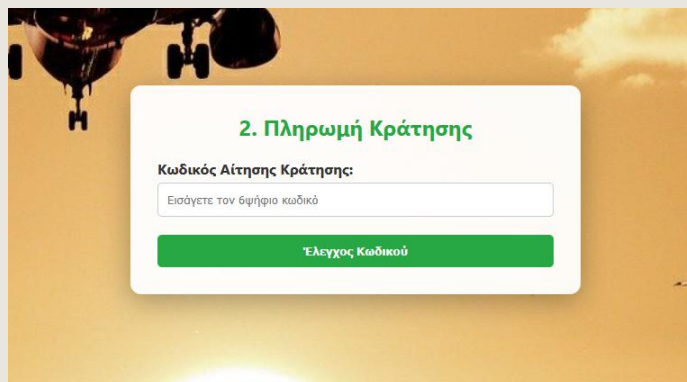


FUNCTION checkApplicationCode()

```
103  ✓ function checkApplicationCode() {  
104      const totalCost = localStorage.getItem('calculatedTotalCost');  
105  ✓  if(document.getElementById('appCode').value === validAppCode){  
106      document.getElementById('paymentSection').style.display = 'block';  
107      document.getElementById('displayCost').innerText = totalCost + '€';  
108  
109      return;  
110  ✓  }else{  
111      alert("Λανθασμένος Κωδικός Αίτησης.");  
112      window.location.replace("../index.html");  
113  }  
114  }
```

FUNCTION executePayment()

```
116 function executePayment() {
117     const cardInput = document.getElementById('cardNumber').value.trim();
118     const cardRegex = /^[0-9]{16}$/;
119     if (!cardRegex.test(cardInput)) {
120         alert("Σφάλμα:  αριθμός της πιστωτικής κάρτας πρέπει να αποτελείται από ακριβώς 16 ψηφία.");
121         return;
122     }else{
123         alert("Μήνυμα Επιβεβαίωσης Ελέγχου Πιστωτικής Κάρτας (eCARDS).\nΜήνυμα Επιβεβαίωσης Πληρωμής.\nΟ κωδικός Check-in σας είναι: " + validCheckinCode);
124         window.location.href = "checkin.html";
125     }
126 }
127
```



➤ **Βήμα 3 (Check-in)** : εδώ ζητείται από τον χρήστη ο κωδικός check-in που δόθηκε κατά την ολοκλήρωση της πληρωμής όπου και ελέγχεται από την [checkCheckinCode\(\);](#)

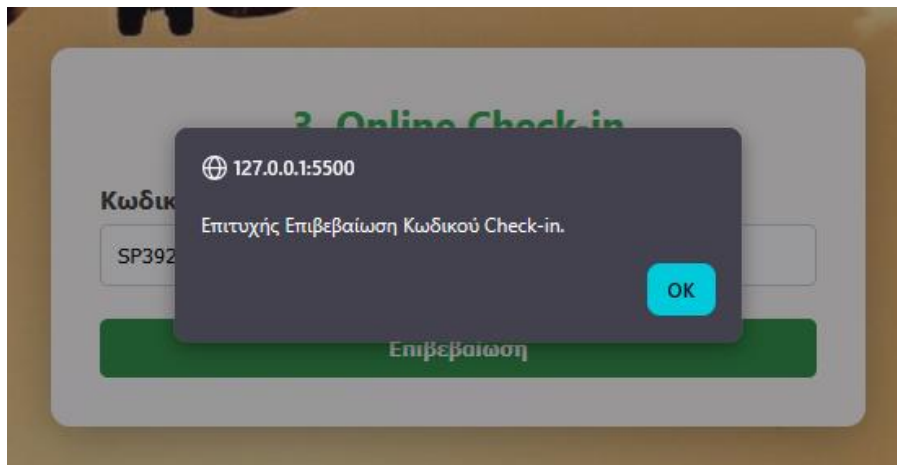
Αν έχουμε επιτυχία εμφανίζεται ο πίνακας επιλογής θέσεων στο αεροπλάνο φτιαγμένος στο css όπου εμφανίζεται μήνυμα αποτυχίας εάν και εφόσον δεν έχουν επιλεγθεί όλες οι θέσεις που πλήρωσε ο χρήστης , έλεγχος μέσω της [finalizeCheckin\(\);](#) και κατά την ολοκλήρωση τερματίζεται η διαδικασία με θετικό μήνυμα και επιστρέφει τον χρήστη στην αρχική σελίδα

FUNCTION checkCheckinCode()

```
128 function checkCheckinCode() {  
129     if(document.getElementById('checkinCode').value === validCheckinCode){  
130         alert("Επιτυχής Επιβεβαίωση Κωδικού Check-in.");  
131         document.getElementById('seatSection').style.display = 'block';  
132         createAirplaneMap();  
133     }else{  
134         alert("Μήνυμα Αποτυχημένης Επιβεβαίωσης Κωδικού Check-in.");  
135         window.location.replace("../index.html");  
136     }  
137 }
```

FUNCTION finalizeCheckin()

```
139  ✓ function finalizeCheckin() {  
140      const required = parseInt(localStorage.getItem('totalRequestedSeats')) || 1;  
141  ✓  if (selectedSeatsArray.length !== required) {  
● 142      alert("Σφάλμα: Πρέπει να επιλέξετε ακριβώς " + required + " θέσεις.");  
143      return;  
144  }  
145  alert("Το ηλεκτρονικό εισιτήριο εκδόθηκε. Η κράτηση είναι ΕΠΙΒΕΒΑΙΩΜΕΝΗ.");  
146  localStorage.clear();  
147  window.location.replace("../index.html");  
148  }
```



3. Online Check-in

Κωδικός Check-in (από SMS):

Επιβεβαίωση

Επιλογή Θέσεων
Επιλέξτε ακριβώς 5 θέσεις.

1A	1B	1C	1D
2A	X	2C	2D
X	3B	X	3D
4A	4B	4C	4D
5A	5B	5C	5D
X	6B	6C	6D
7A	7B	7C	7D
8A	8B	8C	8D
9A	X	9C	9D
10A	10B	10C	10D

Έκδοση Ειστηγρίου

3. Online Check-in

Κωδικός Check-in (από SMS):

Επιβεβαίωση

Επιλογή Θέσεων
Επιλέξτε ακριβώς 5 θέσεις.

1A	1B	1C	1D
2A	X	2C	2D
X	3B	X	3D
X	6B	6C	6D
7A	7B	7C	7D
8A	8B	8C	8D
9A	X	9C	9D
10A	10B	10C	10D

Έκδοση Ειστηγρίου

- 
- **Επικύρωση** : Το σύστημα απορρίπτει λάθος διαβατήρια (π.χ. χωρίς κεφαλαίο γράμμα στην αρχή) ή κάρτες με λιγότερα από 16 ψηφία.
 - **Ευχρηστία** : Προσθήκη Hint Boxes στην αρχική σελίδα για καθοδήγηση του χρήστη.
 - **Συμπέρασμα** : Το πρωτότυπο καλύπτει πλήρως τις προδιαγραφές της εκφώνησης, εξασφαλίζοντας ασφαλή ροή.